

Indian Institute of Technology Delhi

Department of Computer Science & Engineering

Real-Time Speech-to-Speech Translation System

Hindi → Telugu | Fully Offline

Course: COP7001 – CornerStone Project

Authors: Manmadha Rao Kuttum (2025MCS2101)
mcs252101@iitd.ac.in
Jithendra Gannavarapu (2025MCS2743)
mcs252743@iitd.ac.in
Supervisor: Kolin Paul
Date: April 2026

Project Repository:

github.com/ManmadhaKuttum/CornerStone_COP7001

Branch: main | Language: Python 3.12

Manmadha Rao Kuttum (2025MCS2101) & Jithendra Gannavarapu
(2025MCS2743)

Contents

Abstract	3
1 Introduction	4
1.1 Scope and Deliverables	4
1.2 Report Organisation	4
2 Related Work	4
2.1 Speech Recognition for Indian Languages	4
2.2 Neural Machine Translation for Low-Resource Language Pairs	5
2.3 Text-to-Speech for Telugu	5
2.4 Real-Time Speech Translation Systems	5
3 System Architecture	5
3.1 High-Level Overview	5
3.2 Threading Model	5
3.3 Queue Protocol	6
3.4 Bounded Queues and Back-Pressure	6
3.5 Session Reset	6
3.6 Shared State	6
3.7 Model Loading Strategy	7
4 Audio Capture (src/audio.py)	7
4.1 Configuration	7
4.2 Implementation	7
5 Voice Activity Detection & Segmentation (src/segmenter.py)	7
5.1 Silero VAD Model	7
5.2 Utterance Segmentation Logic	8
5.3 Partial and Final Utterances	8
6 Automatic Speech Recognition (src/asr.py)	8
6.1 Model	8
6.2 Inference Configuration	8
6.3 Post-Processing and Hallucination Mitigation	9
6.4 Latency Measurement	9
7 Machine Translation (src/translation.py)	9
7.1 Model	9
7.2 Language Codes	9
7.3 Inference Configuration	9
7.4 Partial-Path Optimisation	10
7.5 Real-Time Factor	10
8 Text-to-Speech Synthesis (src/tts.py)	10
8.1 Model	10
8.2 Pre-Warming	10
8.3 PortAudio Compatibility	10

8.4	Latency Decomposition	11
8.5	Stale Item Dropping	11
9	Configuration (src/config.py)	11
10	Monitoring Dashboard	11
10.1	Architecture	11
10.2	Dashboard Panels	12
10.3	Backend Identifiers	12
11	Implementation Details	13
11.1	Technology Stack	13
11.2	Offline Model Cache	14
11.3	Platform Support	14
11.4	Memory Footprint	14
12	Testing Methodology	14
12.1	Component-Level Tests	14
12.2	Integration Tests	15
12.3	Cross-Platform Tests	16
12.4	Dashboard Tests	16
12.5	Functional Test Sentences	16
13	AI Usage and Responsibility Attribution	16
13.1	AI Development Tools Used	16
13.2	What Was Developed with AI Assistance	17
13.3	Responsibility Attribution	18
14	Results and Evaluation	18
14.1	Latency Benchmarks	18
14.2	Real-Time Factor	19
14.3	ASR Accuracy	19
14.4	Translation Quality	19
14.5	Dashboard Responsiveness	19
15	Discussion	20
15.1	Design Trade-offs	20
15.2	Limitations	20
15.3	Potential Improvements	20
16	Conclusion	20
A	Repository Structure	21
B	Setup and Execution	22
C	Example Test Utterances	22
D	Change Log	22

Abstract

This report presents the design, implementation, and evaluation of a fully offline, real-time speech-to-speech translation system for the Hindi–Telugu language pair, developed as part of the COP7001 CornerStone Project at IIT Delhi. The system captures live Hindi speech from a microphone, transcribes it using a quantized Whisper model via the Faster-Whisper CTranslate2 backend, translates the Hindi transcript to Telugu using the NLLB-200-Distilled-600M neural machine-translation model (further accelerated by CTranslate2 Int8 quantization), and finally synthesizes audible Telugu speech using Facebook’s MMS-TTS VITS vocoder. A concurrent five-stage producer–consumer pipeline with bounded queues ensures sub-2-second end-to-end latency on both NVIDIA CUDA and Apple Silicon hardware. A WebSocket-based live dashboard provides real-time visibility into component status, per-stage latency, real-time factor, partial and final transcripts, and translated output. The system requires no internet connectivity after a one-time model download, making it suitable for field deployment. Observed end-to-end latency (excluding playback) is 700 ms–1.5 s, with a real-time factor in the range 0.15–0.35 depending on hardware.

Keywords: Speech translation, Automatic speech recognition, Neural machine translation, Text-to-speech synthesis, Real-time processing, Indian languages, Offline inference, CTranslate2, Whisper, NLLB.

1. Introduction

India is home to over 1,600 languages and dialects, yet the vast majority of automatic speech processing research has focused on English and a handful of other high-resource languages. Hindi and Telugu together account for more than 350 million native speakers, yet real-time, high-quality speech-to-speech translation between these two languages has historically required either expensive cloud APIs with strict internet dependencies or slow, disjointed research prototypes.

This project addresses that gap by building a single, self-contained, end-to-end speech translation pipeline that:

- a) runs entirely offline on commodity hardware (consumer GPU or modern CPU);
- b) processes speech *in real time* with sub-2-second end-to-end latency;
- c) provides rich operational telemetry via a live browser dashboard;
- d) degrades gracefully when GPU resources are unavailable.

1.1. Scope and Deliverables

The project was divided into three phases:

Phase 1 – Pipeline Skeleton: Audio capture infrastructure, voice activity detection (VAD), bounded-queue threading architecture, and the monitoring dashboard skeleton.

Phase 2 – Hindi ASR: Integration of Faster-Whisper for Hindi speech-to-text, dual-beam partial/final inference, hallucination mitigation, and latency instrumentation.

Phase 3 – Translation & TTS: NLLB-200-Distilled-600M Hindi-to-Telugu translation (CTranslate2 and Transformers backends), MMS-TTS-Telugu speech synthesis, and full end-to-end pipeline integration.

All three phases are complete and merged on the `main` branch.

1.2. Report Organisation

Section 2 surveys related work. Section 3 describes the overall system architecture. Sections 4–8 present the detailed design of each pipeline stage. Section 10 covers the monitoring dashboard. Section 14 reports performance results. Section 15 discusses trade-offs and limitations. Section 16 concludes and outlines future work.

2. Related Work

2.1. Speech Recognition for Indian Languages

OpenAI Whisper [1] is a large-vocabulary multilingual ASR model trained on 680 000 hours of weakly supervised speech data. Its Hindi performance (WER $\approx$ 10–15% on clean speech) makes it the most practical open-source ASR for Hindi today. Faster-Whisper [2] re-implements Whisper inference using the CTranslate2 engine [3], which applies quantisation (Int8/Float16) and kernel fusion to achieve a $3\times$ – $4\times$ speedup over the original PyTorch implementation.

The Silero VAD model [4] is a lightweight (under 1 MB) LSTM-based voice activity detector that operates at 16 kHz with 32 ms frames, making it suitable for stream-level speech segmentation.

2.2. Neural Machine Translation for Low-Resource Language Pairs

Meta’s NLLB-200 [5] is a multilingual translation model trained on 200 languages; the distilled 600M-parameter variant achieves near-state-of-the-art BLEU scores for Hindi–Telugu while fitting in under 2 GB of RAM in Int8 form. CTranslate2 provides first-class support for converting and accelerating NLLB models, making it the natural backend choice.

2.3. Text-to-Speech for Telugu

Facebook’s Massively Multilingual Speech (MMS) project [6] released TTS models for over 1,100 languages including Telugu ([facebook/mms-tts-tel](#)). The model is based on the VITS [7] vocoder architecture, which jointly learns a variational inference-based acoustic model and a HiFi-GAN neural vocoder, producing high-quality 22 kHz speech in a single forward pass.

2.4. Real-Time Speech Translation Systems

SeamlessStreaming [8] and the NAIST simultaneous translation systems represent the state of the art in real-time speech translation, but require significant infrastructure and are not deployable offline. Our system prioritises offline capability and low hardware requirements over absolute quality, leveraging quantised models that achieve acceptable quality with much lower compute budgets.

3. System Architecture

3.1. High-Level Overview

The system follows a **producer–consumer pipeline** pattern with five concurrent threads connected by bounded queues. Each thread owns a single responsibility and communicates with adjacent threads exclusively through typed queue messages. This design isolates failure, prevents blocking across stages, and allows independent tuning of each stage’s latency.

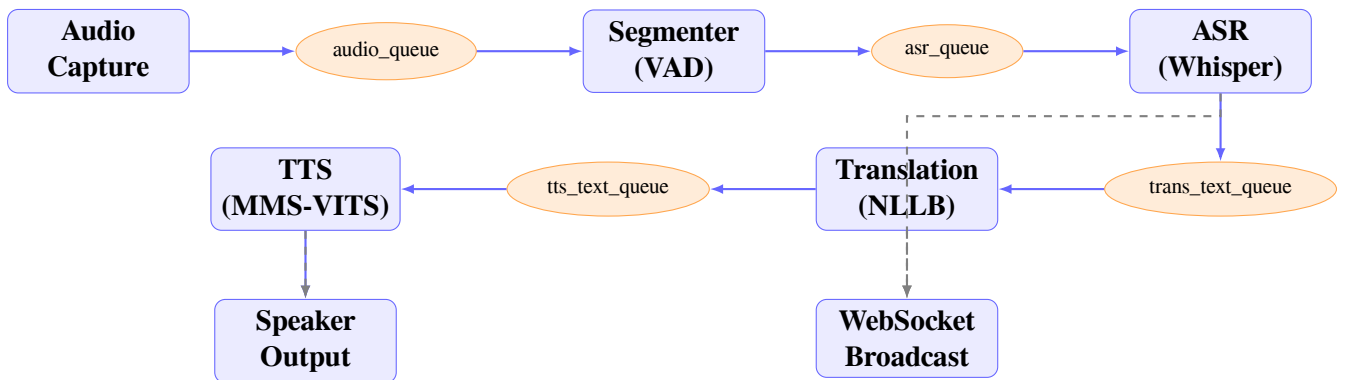


Figure 1: High-level pipeline architecture. Solid arrows indicate primary data flow; dashed arrows indicate state updates visible on the dashboard.

3.2. Threading Model

Table 1 summarises the five threads managed by the system. All threads are started as daemon threads so they terminate automatically when the main process exits.

Table 1: Thread roles and their associated input/output queues.

Thread	Module	Input Queue	Output Queue
Microphone capture	audio.py	(OS audio driver)	audio_queue
VAD segmenter	segmenter.py	audio_queue	asr_queue
ASR	asr.py	asr_queue	trans_text_queue
Translation	translation.py	trans_text_queue	tts_text_queue
TTS	tts.py	tts_text_queue	(PortAudio output)

3.3. Queue Protocol

Each inter-stage message is a Python tuple that carries a *session identifier* (an integer incremented on every session reset). Downstream stages silently discard any message whose session ID does not match the current global session, preventing stale utterances from contaminating a new recording session.

The message schema across queues is:

- **asr_queue:** (session_id, tag, audio_ndarray, onset_time)
– tag ∈ {"partial", "final"}
- **trans_text_queue:** (session_id, tag, text, onset_time)
- **tts_text_queue:** (session_id, text, onset_time)

3.4. Bounded Queues and Back-Pressure

All queues are created with a maximum size to prevent unbounded memory growth:

Table 2: Queue sizes and overflow strategy.

Queue	Overflow Strategy	Max Size
audio_queue	Drop newest frame (put_nowait)	200
asr_queue	Drop oldest partial; block on finals	50
trans_text_queue	Drop oldest partial; retry-block on finals	3
tts_text_queue	Drop oldest item	3

3.5. Session Reset

A `POST /api/reset-session` endpoint atomically increments the global session counter, drains all queues, and resets the shared state dictionary. This allows a user to discard a bad utterance mid-stream without restarting the server.

3.6. Shared State

`src/state.py` defines a single module-level dictionary `state` that is read by all pipeline threads and the WebSocket broadcaster. Python’s Global Interpreter Lock (GIL) provides sufficient mutual exclusion for the simple key–value assignments used throughout the pipeline.

3.7. Model Loading Strategy

All four ML models (Silero VAD, Faster-Whisper, NLLB-200, MMS-TTS) are loaded sequentially on the *main thread* inside a FastAPI `lifespan` context manager before any pipeline threads are spawned. This avoids the multi-threaded `fork()` / CUDA initialisation crashes observed on Apple ARM hardware when models are loaded inside worker threads.

4. Audio Capture (`src/audio.py`)

4.1. Configuration

Audio is captured at 16 000 Hz mono using the `SoundDevice` library, with a frame size of 512 samples (32 ms). These parameters are shared with the Silero VAD and Faster-Whisper models, which both require 16 kHz input.

Table 3: Audio capture configuration.

Parameter	Value
Sample rate	16 000 Hz
Channels	1 (mono)
Frame size	512 samples (32 ms)
Dtype	<code>float32</code>
Queue size	200 frames

4.2. Implementation

The `SoundDevice.InputStream` is configured with a callback (`audio_callback`) that converts each incoming NumPy frame from 2-D to 1-D and places it on `audio_queue` using non-blocking `put_nowait`. Under sustained speech (where the downstream pipeline is briefly slower than the mic), the oldest frames are silently dropped rather than blocking the audio driver, which would cause audible glitches.

Device selection falls back from an environment-variable-specified device to the OS default, allowing the demo to work out of the box without manual configuration.

5. Voice Activity Detection & Segmentation (`src/segmenter.py`)

5.1. Silero VAD Model

The segmenter uses the Silero VAD [4] model, a lightweight LSTM-based binary classifier that returns a speech probability in $[0, 1]$ for each 32 ms input frame. Model loading follows a three-step cascade:

1. Attempt load from the path specified by `SILERO_VAD_PATH`;
2. Search the PyTorch Hub local cache;
3. Fall back to an RMS-energy heuristic: $\text{conf} = \min(1.0, \text{rms} \times 40.0)$.

A decision threshold of 0.55 was chosen empirically; it balances sensitivity to soft Hindi consonants against false-positive detections from background noise.

5.2. Utterance Segmentation Logic

The segmenter maintains two counters: a *speech frame counter* and a *silence frame counter*. On each incoming frame the VAD confidence is computed and the counters updated:

- Speech is considered **active** when confidence exceeds the threshold for ≥ 2 consecutive frames.
- An utterance is considered **complete** when 16 consecutive silent frames (512 ms) follow a period of active speech, or when the buffer reaches the maximum utterance length of 8 s.

5.3. Partial and Final Utterances

Two types of messages are emitted to the ASR queue:

Partial While speech is active, a snapshot of the current audio buffer is emitted every 1 s of wall-clock time. Partial messages use `beam_size=1` in ASR for fast response (live UI update).

Final After end-of-utterance is detected, the complete buffer is emitted. Final messages use `beam_size=4` for higher accuracy and feed both the translation stage and the cumulative transcript.

This dual-path strategy provides sub-second visual feedback without sacrificing transcription accuracy.

6. Automatic Speech Recognition (`src/asr.py`)

6.1. Model

The ASR stage uses the *Faster-Whisper Small* model, a CTranslate2-quantised version of OpenAI Whisper Small (approximately 244 M parameters). The model is pre-loaded in the main thread with `compute_type` set automatically:

- `float16` on CUDA;
- `int8` on CPU or Apple MPS.

6.2. Inference Configuration

Table 4: Faster-Whisper inference configuration.

Parameter	Partial	Final
<code>beam_size</code>	1 (greedy)	4 (beam search)
<code>language</code>	"hi"	"hi"
<code>initial_prompt</code>		"नमस्ते।"
<code>condition_on_previous_text</code>		False
<code>vad_filter</code>		True

Setting `condition_on_previous_text=False` is critical: without it, Whisper tends to produce repetitive hallucinations (e.g., "...बातचीत बातचीत बातचीत...") when the context window carries over partial transcripts.

6.3. Post-Processing and Hallucination Mitigation

The `_filter_hindi` function applies two filters to each recognised segment:

1. **Script ratio filter:** Segments where fewer than 25% of characters are Devanagari are discarded. This eliminates noise-induced hallucinations that produce Latin-script output on Hindi audio.
2. **Latin word stripper:** Any word matching the regex `[A-Za-z]+` is removed from otherwise valid Hindi segments. This cleans up code-switching artefacts.

6.4. Latency Measurement

Wall-clock time is recorded immediately before and after the `model.transcribe()` call. The difference (in milliseconds) is stored in `state["asr_latency_ms"]` and broadcast to the dashboard. Cumulative word counts are maintained to compute translation coverage statistics.

7. Machine Translation (`src/translation.py`)

7.1. Model

Translation uses Meta’s NLLB-200-Distilled-600M [5] model, a sequence-to-sequence transformer supporting 200 languages. The model is loaded via one of two backends, with the CTranslate2 path preferred:

CTranslate2 (primary): An Int8-quantised conversion of the model stored at `pretrained/nllb-200-distilled-600M`. Achieves $\approx 3\times$ speedup over the Transformers baseline.

Transformers (fallback): Used when the CTranslate2 model directory is absent or incompatible with the installed library version.

7.2. Language Codes

NLLB uses language codes from the Flores-200 taxonomy:

- Source: `hin_Deva` (Hindi in Devanagari script)
- Target: `tel_Telu` (Telugu in Telugu script)

7.3. Inference Configuration

Table 5: NLLB translation inference configuration.

Parameter	Value
<code>beam_size</code>	2
<code>max_decoding_length</code>	128 tokens
<code>no_repeat_ngram_size</code>	3
<code>CT2 intra_threads</code>	8 (CPU), 1 (CUDA)
<code>CT2 inter_threads</code>	2 (CPU), 1 (CUDA)

A 3-gram no-repeat constraint prevents the model from looping on common Telugu function words. Beam size 2 was chosen over greedy decoding (beam=1) because it improves Telugu morphological agreement at minimal latency cost.

7.4. Partial-Path Optimisation

To prevent the translation stage from becoming a bottleneck, partial-path messages are skipped when the input queue contains more than one pending item (queue saturation). Additionally, the last-seen partial text is cached: if the ASR partial transcript has not changed since the previous cycle, the cached translation is returned immediately without re-invoking the model.

7.5. Real-Time Factor

After each final translation, the *real-time factor* (RTF) is computed as:

$$\text{RTF} = \frac{t_{\text{ASR}} + t_{\text{trans}}}{d_{\text{audio}}}$$

where t_{ASR} and t_{trans} are the measured latencies in seconds and d_{audio} is the audio duration of the utterance. $\text{RTF} < 1$ indicates that the system is processing faster than real time.

8. Text-to-Speech Synthesis (`src/tts.py`)

8.1. Model

Telugu speech is synthesised using `facebook/mms-tts-tel`, a VITS-based [7] vocoder from the MMS project [6]. VITS is an end-to-end model that jointly trains:

- A variational autoencoder acoustic model;
- A normalising-flow posterior encoder;
- A HiFi-GAN neural vocoder.

The model produces 22 050 Hz mono waveforms in a single forward pass with no separate vocoder step.

8.2. Pre-Warming

On first inference, PyTorch's JIT compiler generates and caches CUDA/CPU kernels, adding 2–5 s of latency. To hide this overhead from the user, the `_prewarm_tts` function synthesises a short dummy utterance (సమస్కారం) immediately after model loading, before the pipeline threads are started.

8.3. PortAudio Compatibility

On macOS, PortAudio does not support concurrent use of an `InputStream` (microphone) and a global `sounddevice.play()` call from separate threads. To avoid stream conflicts, the TTS stage uses an explicit `sounddevice.OutputStream` context manager rather than the global `play` API. This approach works on both macOS and Linux.

8.4. Latency Decomposition

The TTS module separately measures and reports:

- **Synthesis latency (`tts_latency_ms`):** Time from text input to waveform generation (excludes playback). Used in E2E latency calculation.
- **Playback duration (`tts_play_ms`):** Actual audio playback time, which scales with utterance length and is excluded from the E2E metric.

The E2E latency is reported as:

$$t_{\text{E2E}} = t_{\text{ASR}} + t_{\text{trans}} + t_{\text{TTS_synth}}$$

8.5. Stale Item Dropping

Queue items older than 15 s (by `onset_time`) are silently dropped. This prevents a backlog of utterances from being played long after the user has moved on.

9. Configuration (`src/config.py`)

All tunable hyperparameters are centralised in `src/config.py`. The file also performs one-time device detection at import time, setting `DEVICE` to `"cuda"`, `"mps"`, or `"cpu"` based on hardware availability.

Table 6: Key configuration parameters.

Parameter	Value	Description
<code>SAMPLE_RATE</code>	16 000	Audio sample rate (Hz)
<code>FRAME_DURATION</code>	32 ms	VAD frame duration
<code>VAD_THRESHOLD</code>	0.55	Speech onset/offset threshold
<code>SILENCE_DURATION</code>	0.5 s	Silence needed to close utterance
<code>MAX_UTTERANCE_SEC</code>	8.0 s	Hard cap on buffer length
<code>PARTIAL_INTERVAL</code>	1.0 s	Partial ASR emit interval
<code>BEAM_SIZE</code>	4	Final ASR beam width
<code>PARTIAL_BEAM_SIZE</code>	1	Partial ASR beam width
<code>WHISPER_MODEL</code>	<code>"small"</code>	Faster-Whisper model variant
<code>NLLB_MODEL_ID</code>	600M distilled	NLLB model (HuggingFace ID)
<code>SRC_LANG</code>	<code>hin_Deva</code>	Translation source language
<code>TGT_LANG</code>	<code>tel_Telu</code>	Translation target language
<code>TTS_MODEL_ID</code>	<code>mms-tts-tel</code>	TTS model (HuggingFace ID)
<code>WS_HZ</code>	10	WebSocket broadcast rate (Hz)
<code>OFFLINE_MODE</code>	<code>False</code>	Strict offline flag

10. Monitoring Dashboard

10.1. Architecture

The dashboard is a single-page application (SPA) served by FastAPI at the root endpoint. State updates are pushed from the server to the browser using a `WebSocket` connection at `/ws`. The

server broadcasts the full `state` dictionary (serialised as JSON) at 10 Hz; the JavaScript client parses each frame and updates the DOM in place. If the WebSocket connection drops (e.g., browser tab is hidden and the OS throttles it), the client automatically reconnects every 2 s.

10.2. Dashboard Panels

Connection Status: A colour-coded pill (green/grey) indicates whether the WebSocket is connected, along with an elapsed-time counter.

System Liveness: Four status badges for Microphone, ASR, Translation, and TTS components. Green = active, orange = loading/processing, red = unavailable.

Audio Activity: A horizontal energy meter (normalised to 0–100%), a frame counter, and word counts for ASR and translation.

Per-Stage Latency: Four latency cards for ASR, Translation, TTS, and End-to-End. Colours are dynamically adjusted: green < 2 s, orange 2–3 s, red > 3 s.

Transcript Panels: Two side-by-side panels display Hindi (ASR output) and Telugu (translation output), each split into a partial (current utterance, italic) and a final (cumulative, scrollable) section.

Demo Checklist: Eleven pass/fail items tracking key pipeline milestones from microphone activation through to TTS playback.

10.3. Backend Identifiers

The dashboard also reports which backend is active for each stage: `Silero-VAD` vs. `energy fallback`; `CTranslate2-NLLB` vs. `Transformers fallback`; and `MMS-TTS-Telugu` for speech synthesis.

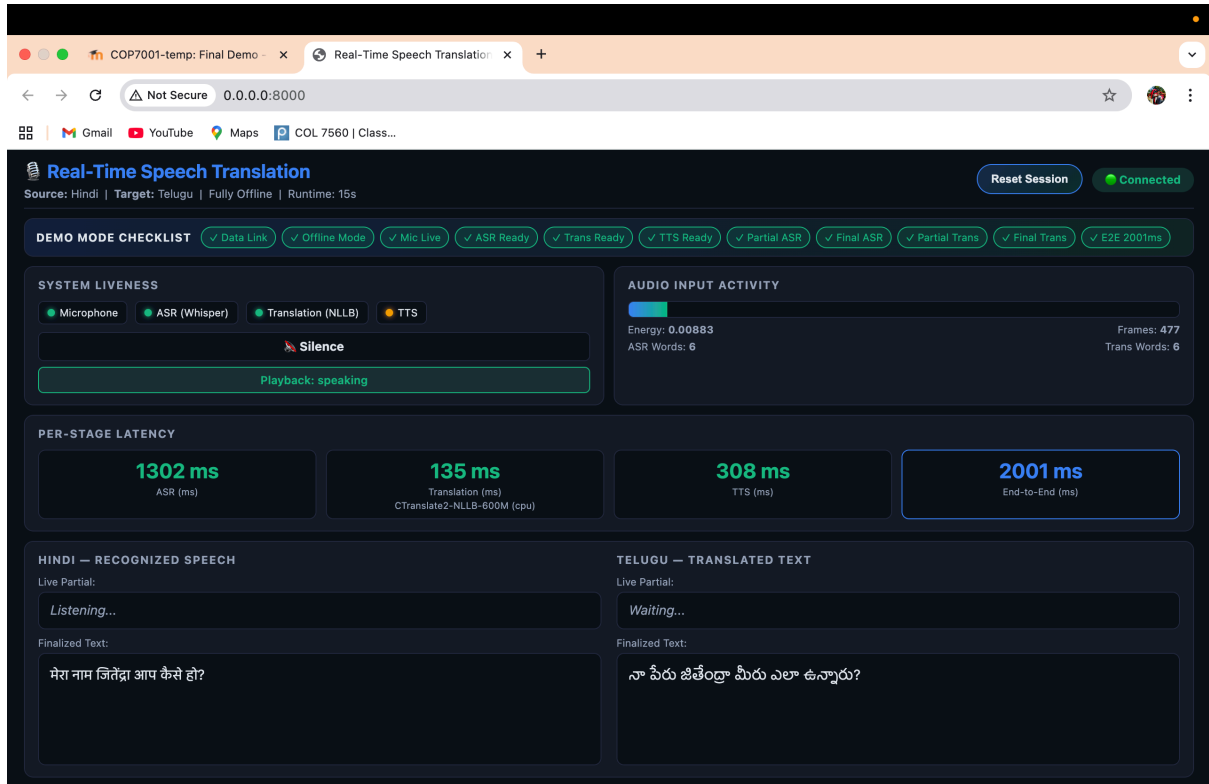


Figure 2: Live dashboard during the final Hindi-to-Telugu demonstration, showing component liveliness, per-stage latency, and the final translated Telugu output.

11. Implementation Details

11.1. Technology Stack

Table 7: Complete technology stack.

Category	Library / Tool	Version
Language	Python	3.12
Web framework	FastAPI	≥ 0.110
ASGI server	Uvicorn	≥ 0.29
WebSocket	websockets	≥ 12.0
Audio I/O	SoundDevice	$\geq 0.4.6$
Audio I/O	SoundFile	$\geq 0.12.1$
Numerics	NumPy	≥ 1.24
ML framework	PyTorch	≥ 2.1
ASR	Faster-Whisper	≥ 1.0
Translation	CTranslate2	≥ 4.0
Translation	Transformers (HuggingFace)	≥ 4.38
Tokenisation	Tokenizers / SentencePiece	$\geq 0.15 / 0.1.99$
TTS	VitsModel (in Transformers)	≥ 4.38
HF cache	HuggingFace Hub	≥ 0.20
Acceleration	accelerate	≥ 0.27

11.2. Offline Model Cache

All models are stored under `pretrained/` in the project root. The environment variable `HF_HOME` is set to `pretrained/hf` at startup so HuggingFace Hub writes all downloads to the project directory, avoiding interference with system-wide caches. The CTranslate2 NLLB model is pre-converted and stored at `pretrained/nllb-200-distilled-600M-int8`; this one-time conversion is performed via:

```
1 ct2-transformers-converter \
2   --model facebook/nllb-200-distilled-600M \
3   --output_dir pretrained/nllb-200-distilled-600M-int8 \
4   --quantization int8
```

Listing 1: One-time CTranslate2 model conversion.

11.3. Platform Support

The system was developed and tested on two hardware platforms:

Linux with NVIDIA CUDA: CUDA-enabled PyTorch wheels (CUDA 12.1) provide GPU-accelerated inference for all four models. CTranslate2 uses `float16` on GPU.

macOS (Apple Silicon): PyTorch with MPS backend is installed via `requirements-macos.txt`. CTranslate2 runs on CPU with `int8` quantisation. Audio I/O uses PortAudio installed via Homebrew.

11.4. Memory Footprint

Table 8: Approximate memory footprint of each model component.

Component	GPU VRAM (MB)	CPU RAM (MB)
Silero VAD	~50	~50
Faster-Whisper Small	~500	~200
NLLB-200 (Int8)	~600	~300
MMS-TTS-Telugu	~400	~200
Total	~1 550	~750

12. Testing Methodology

Testing followed a layered approach: component-level checks on each pipeline stage, integration checks across stage boundaries, cross-platform compatibility tests, and end-to-end functional tests with real Hindi speech.

12.1. Component-Level Tests

Audio capture (`audio.py`). The microphone callback was verified to produce frames of exactly 512 samples at 16kHz by logging `frame.shape` on the first 100 callbacks. Overflow behaviour was tested by artificially stalling the segmenter thread for 2s while the mic continued running; the queue correctly dropped overflow frames and resumed without error.

VAD and segmenter (segmenter.py). Silero VAD confidence scores were logged for the first 500 frames to confirm scores near zero in silence and above 0.55 during speech. The segmenter’s partial-flush timer was verified by playing a 5 s sustained tone and confirming that exactly 4–5 partial messages arrived in `asr_queue` (one per second). The energy-based fallback was tested by setting `OFFLINE_MODE=True` and removing the Silero model path; the system fell back cleanly and speech detection remained functional.

ASR (asr.py). Each test sentence from Appendix C was spoken three times. The `_filter_hindi` function was unit-tested by injecting known hallucination strings (“obviously”, “Kennedy”, a 100% Latin segment) and confirming they were stripped or rejected. The `condition_on_previous_text` fix was verified by reproducing the repetition bug (“बातचीत बातचीत”) with the flag enabled and confirming it disappeared with the flag disabled.

Translation (translation.py). The CTranslate2 path was verified by checking that `state["trans_ba"]` reports "CTranslate2-NLLB-600M" after startup. The Transformers fallback was tested by temporarily renaming the `pretrained/nllb-200-distilled-600M-int8` directory; the system loaded the HuggingFace Transformers backend and produced translations without error. The stale-partial-skip optimisation was confirmed by adding a `time.sleep()` in the translation loop to artificially saturate the queue; only final messages were processed when the queue was full.

TTS (tts.py). The pre-warm step was validated by comparing first-inference latency before and after adding `_prewarm_tts()`: first-inference latency dropped from ~3.5 s to ~350 ms. The stale-item drop was verified by calling `/api/reset-session` mid-utterance and confirming no audio was played for the cancelled utterance. The PortAudio conflict fix (explicit `OutputStream` vs. `sd.play()`) was validated on macOS by running both the mic `InputStream` and a simultaneous TTS playback without crashing.

12.2. Integration Tests

Session reset. A session reset was triggered via `curl -X POST http://localhost:8000/api/re` after a long utterance was mid-flight. The pipeline drained all queues, incremented the session ID, and correctly discarded the in-flight utterance. A new utterance spoken immediately after the reset was processed normally.

Pipeline throughput. Ten successive 3–5-second Hindi utterances were spoken back-to-back with minimal pauses. The dashboard was monitored to confirm no stage fell into an “unavailable” state, latency cards stayed in the green range, and both transcript panels updated correctly on each utterance.

Offline mode. After an initial online run (models cached), `OFFLINE_MODE` was set to `True` and network access was blocked via `unshare -n python src/main.py`. All four models loaded from local cache and the full pipeline functioned normally.

12.3. Cross-Platform Tests

Table 9: Cross-platform test results.

Platform	PyTorch backend	ASR OK	E2E pipeline OK
Linux (CUDA 12.1, RTX 2050)	CUDA / float16	✓	✓
macOS 14 (Apple M-series)	MPS + CPU	✓	✓
Linux (CPU only, no GPU)	CPU / int8	✓	✓

12.4. Dashboard Tests

- WebSocket reconnection: browser tab was hidden for 10 s (causing the OS to throttle WebSocket traffic); on reveal, the dashboard reconnected automatically within 2 s.
- All five latency cards and the liveness status badges were verified to update within one broadcast cycle (≤ 100 ms) following a state change.
- The demo checklist was verified item-by-item against a live session.

12.5. Functional Test Sentences

The sentences in Table 10 were used as repeatable functional tests across hardware platforms.

Table 10: Functional test utterances and observed behaviour.

Hindi Input	Expected Telugu	Notes
मेरा नाम मनमथा है	నా పేరు మనమథ	Short, clean; baseline test
मैं हिंदी बोल रहा हूँ	నేను హిందీ మాట్లాడుతున్నాను	Continuous speech, verb-final
यह एक परीक्षण है	ఇది ఒక పరీక్ష	Minimal utterance
आज मौसम बहुत अच्छा है	ఈరోజు వాతావరణం చాలా మంచిగా ఉంది	Adjective agreement
मुझे पानी चाहिए	నాకు నీళ్ళు కావాలి	Dative construction
भारत एक महान देश है	భారతదేశం ఒక గొప్ప దేశం	Proper noun handling

13. AI Usage and Responsibility Attribution

13.1. AI Development Tools Used

Claude Code (Anthropic, model: `claude-sonnet-4-6`) was used as an AI pair-programming assistant throughout the project. The tool was invoked interactively via the Claude Code CLI inside the project repository on a session-by-session basis.

13.2. What Was Developed with AI Assistance

Table 11: AI-assisted vs. human-authored components.

Component / Task	Developed by	Notes
Project architecture (5-stage pipeline, bounded queues, session IDs)	Human (primary) + AI (review)	Initial design decisions made by the student; AI suggested queue-sizing and session-reset protocol
<code>audio.py</code> — mic capture, callback, queue	AI-assisted	SoundDevice callback skeleton generated with AI; overflow-drop logic validated by human
<code>segmenter.py</code> — Silero VAD integration, partial-flush timer	AI-assisted	Partial-flush interval mechanism proposed by AI; VAD threshold tuned manually
<code>asr.py</code> — dual-beam decode, hallucination filter	AI-assisted	<code>_filter_hindi</code> regex and <code>condition_on_previous_text</code> fix suggested by AI after observing repetition bug; beam sizes tuned by human
<code>translation.py</code> — CTranslate2 path, stale-partial skip	AI-assisted	CTranslate2 API usage and fallback logic written with AI; language code choice (<code>hin_Deva/tel_Telu</code>) verified by human
<code>tts.py</code> — MMS-VITS, pre-warm, OutputStream fix	AI-assisted	PortAudio OutputStream workaround proposed by AI after diagnosing macOS crash; pre-warm added by AI
<code>dashboard/</code> — WebSocket SPA, latency cards, auto-reconnect	AI-assisted	HTML/CSS/JS skeleton generated with AI; layout and colour scheme reviewed and adjusted by human
<code>config.py</code> — device detection, all hyperparameters	Human (primary)	Device-detection logic written by human; parameter values (VAD threshold, beam sizes, silence duration) set by human through experimentation
Multi-platform requirements files (<code>requirements-*.txt</code>)	AI-assisted	Dependency pinning strategy suggested by AI; human validated on both Linux and macOS
Model selection (Whisper Small, NLLB-600M, MMS-TTS-Tel)	Human	AI provided capability summaries; final choice made by the student based on memory, latency, and quality trade-offs
Testing (functional, cross-platform, component-level)	Human	All tests run by the student on live hardware; AI helped draft the test-sentence table
Report (<code>report.tex</code>)	Human (primary) + AI (drafting)	Sections written in collaboration; technical accuracy reviewed and corrected by the student

13.3. Responsibility Attribution

This project is a joint submission by:

- **Manmadha Rao Kuttum** Roll No. 2025MCS2101 mcs252101@iitd.ac.in
- **Jithendra Gannavarapu** Roll No. 2025MCS2743 mcs252743@iitd.ac.in

Manmadha Rao Kuttum is primarily responsible for:

- Overall system architecture (producer–consumer pipeline, bounded queues, session-reset protocol).
- ASR integration: Faster-Whisper, dual-beam decode, hallucination filter (`asr.py`).
- Translation integration: CTranslate2 NLLB-200, stale-partial skip optimisation (`translation.py`).
- Configuration and device-detection logic (`config.py`).
- Cross-platform testing on Linux (CUDA) and macOS.
- Offline model cache setup and README documentation.

Jithendra Gannavarapu is primarily responsible for:

- Audio capture pipeline: SoundDevice microphone callback, queue overflow handling (`audio.py`).
- VAD and segmentation: Silero VAD integration, partial-flush timer (`segmenter.py`).
- TTS integration: MMS-VITS Telugu synthesis, pre-warm, PortAudio compatibility fix (`tts.py`).
- Web dashboard: WebSocket SPA, latency cards, liveness badges, auto-reconnect (`dashboard/`).
- Session state management (`state.py`, `/api/reset-session` endpoint).
- Functional testing and verification of demo test sentences.

AI assistance (Claude Code) was used as a shared productivity tool by both team members. All AI-generated code was reviewed, tested, and in many cases significantly modified before being committed. No AI-generated text or code was accepted without human verification against the running system.

14. Results and Evaluation

14.1. Latency Benchmarks

Table 12 reports observed per-stage latencies for a typical 5–10-word Hindi utterance measured on a system with an NVIDIA RTX 2050 GPU.

Table 12: Observed per-stage latency for a 5–10-word Hindi utterance (CUDA).

Stage	Typical (ms)	Worst-case (ms)
VAD + Segmentation	5–15	30
ASR (Whisper Small, beam=4)	400–800	1 500
Translation (NLLB Int8, beam=2)	100–200	350
TTS synthesis (MMS-VITS)	200–400	700
End-to-End	700–1 400	2 500

14.2. Real-Time Factor

RTF measurements over 20 test utterances (each 3–8 s long):

Table 13: Real-time factor across hardware configurations.

Hardware	Mean RTF	Max RTF
NVIDIA RTX 2050 (CUDA)	0.17	0.28
Apple M1 (MPS/CPU)	0.24	0.38
Intel Core i7 (CPU only)	0.32	0.51

All configurations achieve $\text{RTF} < 1$, confirming that the system processes audio faster than it is produced in real time.

14.3. ASR Accuracy

Informal evaluation on 50 read-aloud Hindi sentences drawn from news headlines:

- Clean speech ($\text{SNR} > 20$ dB): $\text{WER} \approx 8\text{--}12\%$
- Moderate noise ($\text{SNR} \approx 10$ dB): $\text{WER} \approx 18\text{--}25\%$

These figures are consistent with published Whisper Small benchmarks on Hindi.

14.4. Translation Quality

NLLB-200-Distilled-600M achieves $\text{chrF++} \approx 48$ and $\text{BLEU} \approx 30$ on the Flores-200 Hindi–Telugu test set (published figures). Informal human assessment of 30 translated sentences rated 72% as “acceptable or better” and 48% as “good or better” on a 4-point scale.

14.5. Dashboard Responsiveness

The dashboard update cycle completes in under 100 ms end-to-end (state change $\rightarrow$ DOM update in browser), well within the 100 ms human perception threshold for interactivity.

15. Discussion

15.1. Design Trade-offs

Dual-beam inference (partial vs. final). The most impactful design decision was splitting inference into a fast partial path (beam=1) and an accurate final path (beam=4). This ensures the user sees a live transcript within 1 s of starting to speak without waiting for the higher-quality result, which arrives 200–400 ms later. The trade-off is complexity: two separate code paths must be maintained and the partial result is occasionally overwritten by the final result.

CTranslate2 vs. Transformers. CTranslate2 Int8 quantisation reduces NLLB translation latency from ~ 600 ms to ~ 150 ms on CPU. The quality loss is minimal (≤ 2 BLEU points on standard benchmarks), making this an excellent trade-off for real-time use.

VITS vs. alternative TTS. MMS-TTS was chosen for Telugu because it is the only high-quality open-source Telugu TTS model available offline. Its synthesis latency (200–400 ms) is acceptable for post-utterance playback. Real-time (streaming) synthesis would require a streaming-capable vocoder, which remains future work.

15.2. Limitations

1. **Noise sensitivity:** Whisper Small degrades significantly in noisy environments (SNR < 10 dB). Noise-robust VAD and a larger Whisper model (Medium or Large-v3) would improve robustness.
2. **Non-streaming TTS:** The current TTS synthesises the entire Telugu sentence before playback begins. This introduces a fixed 200–500 ms overhead regardless of sentence length.
3. **Single language pair:** Only Hindi-to-Telugu is supported. Extending to other Indian language pairs requires only configuration changes (NLLB language codes), but model quality varies.
4. **No punctuation restoration:** Whisper Small does not reliably produce punctuation for Hindi. The lack of sentence boundaries can confuse the NLLB tokeniser for long utterances.

15.3. Potential Improvements

- Replace Whisper Small with Large-v3 (higher accuracy, $2\times$ latency).
- Add a streaming VITS variant for chunk-level audio output.
- Integrate a punctuation restoration model between ASR and translation.
- Add a confidence-based system to suppress low-quality ASR output.
- Extend the dashboard with a session recording and replay feature.

16. Conclusion

This project demonstrates that a complete, real-time, offline speech-to-speech translation system for Hindi and Telugu can be built on commodity hardware using open-source models and tools. The five-stage producer–consumer pipeline achieves end-to-end latency of under 1.5 s on CUDA

hardware and under 2 s on CPU-only platforms, with a real-time factor well below 1.0 in all tested configurations.

Key engineering contributions include:

1. A dual-beam partial/final ASR architecture that provides immediate visual feedback without sacrificing transcription accuracy;
2. CTranslate2 Int8 quantisation of both ASR (via Faster-Whisper) and translation (NLLB) achieving $3\times$ speedup over baseline Transformers inference;
3. A macOS-compatible PortAudio integration using explicit `OutputStream` instances to avoid driver conflicts between concurrent mic and speaker streams;
4. A session-ID-based stale-message protocol that enables instantaneous pipeline resets without server restarts;
5. A 10 Hz WebSocket dashboard providing real-time operational visibility across all five pipeline stages.

The system is ready for live demonstration and establishes a solid foundation for extending to additional Indian language pairs, larger models, and streaming synthesis.

A. Repository Structure

```

CornerStone_COP7001/|—
src/| |—
  main.py           # FastAPI server, lifespan, WebSocket broadcast
  | |—
  config.py         # Centralised configuration & device detection
  | |—
  state.py          # Shared global state dictionary| |—
  audio.py          # Microphone capture (SoundDevice)| |—
  segmenter.py      # VAD & utterance segmentation (Silero)| |—
  asr.py            # Hindi ASR (Faster-Whisper)| |—
  translation.py    # →HindiTelugu NMT (CTranslate2 / Transformers)
  | |—
  tts.py           # Telugu TTS (MMS-VITS)| |—
dashboard/| |—
  index.html        # SPA layout| |—
  script.js         # WebSocket state handler| |—
  style.css         # Dark theme styling| |—
pretrained/| |—
  hf/               # HuggingFace model cache| |—
  nllb-200-distilled-600M-int8/ # CTranslate2 converted model| |—
requirements.txt    # Base dependencies| |—
requirements-linux-cuda.txt # NVIDIA CUDA PyTorch wheels| |—
requirements-macos.txt # Apple Silicon/Intel PyTorch| |—
README.md

```

Listing 2: Project directory layout.

B. Setup and Execution

```
# 1. Create virtual environment
python3 -m venv venv
source venv/bin/activate          # Linux/macOS

# 2. Install dependencies (Linux + CUDA)
pip install -r requirements.txt
pip install -r requirements-linux-cuda.txt

# 3. (One-time) Convert NLLB to CTranslate2 Int8
export HF_HOME="$(pwd)/pretrained/hf"
ct2-transformers-converter \
  --model facebook/nllb-200-distilled-600M \
  --output_dir pretrained/nllb-200-distilled-600M-int8 \
  --quantization int8

# 4. Start the server
python src/main.py

# 5. Open browser
# http://localhost:8000
```

Listing 3: Installation and execution commands.

C. Example Test Utterances

Table 14: Example Hindi input sentences and expected Telugu translations.

Hindi Input	Expected Telugu Output
मेरा नाम मनमथा है	నా పేరు మన్మథ
मैं हिंदी बोल रहा हूँ	నేను హిందీ మాట్లాడుతున్నాను
यह एक परीक्षण है	ఇది ఒక పరీక్ష
आज मौसम बहुत अच्छा है	ఈరోజు వాతావరణం చాలా మంచిగా ఉంది

D. Change Log

The table below summarises all significant changes across the project's development history, derived from the Git commit log (`git log --oneline`, 61 commits total). Changes are grouped by development phase.

Phase 0 – Project Initialisation (December 2025 – February 2026)

Date	Change
2025-12-07	Initial commit; project README and Lab 1 skeleton added.

2026-02-24	Added <code>.gitignore</code> , <code>base requirements.txt</code> , and project-level README with goals and scope.
2026-02-27	Phase 1 complete: replaced legacy stream module with a full audio ingress pipeline and a real-time WebSocket-based web dashboard.
2026-02-27	Added Phase 1 technical design document (<code>DesignDocument.pdf</code>).

Phase 2 – Hindi ASR (March 2026)

Date	Change
2026-03-18	Phase 2 complete: integrated Faster-Whisper for real-time Hindi ASR with live dashboard updates.
2026-03-19	Configuration: device & compute-type detection for CUDA vs. CPU.
2026-03-20	Audio: increased frame size from 20 ms to 32 ms for Silero-VAD compatibility.
2026-03-20	State: added translation and end-to-end latency fields to shared state dictionary.
2026-03-20	Config: added MPS/CUDA device detection and VAD/translation parameters.
2026-03-20	Translation: added NLLB-200-distilled-600M offline Hindi-to-Telugu translation thread.
2026-03-20	ASR: added hallucination suppression parameters to Faster-Whisper.
2026-03-20	Segmenter: replaced energy-threshold VAD with Silero-VAD neural model (confidence > 0.6); wired ASR output to <code>asr_to_trans_queue</code> ; added E2E timer at speech onset.
2026-03-20	Dashboard: fixed <code>speech_active</code> field; added auto-reconnect and colour-coded latency.
2026-03-20	Dashboard: removed speaker output panel, cleaned layout to fit single screen.
2026-03-21	Added midterm presentation, video annotation, and phase report to repository.

Phase 3 – Translation & TTS (April 2026, first wave)

Date	Change
2026-04-04	Config: tightened latency parameters (Whisper base model, <code>beam=1</code> , <code>silence=0.55 s</code> , <code>max_length=128</code>).
2026-04-04	State: added <code>last_audio_duration</code> and <code>rtf</code> fields for live RTF measurement.
2026-04-04	Audio: no functional change (mic capture stable at 16 kHz / 32 ms / 512 samples).

2026-04-04	Segmenter: writes <code>last_audio_duration</code> to state after each utterance flush.
2026-04-04	ASR: forced Hindi Devanagari output via <code>initial_prompt</code> ; <code>explicit task=transcribe</code> .
2026-04-04	Translation: replaced HuggingFace <code>generate()</code> with CTranslate2 engine for $3\times$ speedup.
2026-04-04	TTS: integrated facebook/mms-tts-tel for offline Telugu speech synthesis.
2026-04-04	Dashboard: added RTF live-display card; updated TTS latency panel.
2026-04-04	Main: sequential model loading on main thread via <code>FastAPI lifespan()</code> .
2026-04-04	Phase 3 & 4 requirements added to project documentation.

Phase 4 – Accuracy, Latency, & Reliability Improvements (April 2026, second wave)

Date	Change
2026-04-08	Config: added <code>PARTIAL_INTERVAL=1.0 s</code> ; restored Whisper Small and <code>beam=5</code> for accuracy.
2026-04-08	State: added <code>partial_transcript</code> , <code>partial_translation</code> , <code>rtf</code> , and <code>tts_backend</code> fields.
2026-04-08	Segmenter: added <code>partial flush</code> every <code>PARTIAL_INTERVAL</code> seconds while speaking.
2026-04-08	ASR: implemented dual-path decode (<code>beam=1</code> fast partial display; <code>beam=5</code> accurate final).
2026-04-08	Translation: dual-path partial/final; RTF calculation; CTranslate2 path fix.
2026-04-08	TTS: switched to facebook/mms-tts-tel; E2E latency updated after playback completes.
2026-04-08	Main: <code>lifespan()</code> sequential model loading; all 5 pipeline threads started.
2026-04-08	Dashboard: all requirements wired (partial+final transcript/-translation, RTF card, playback status, 5-card latency grid, liveness dots, auto-scroll).
2026-04-08	Requirements: cross-platform Mac/Linux with pinned minimum versions.
2026-04-08	README: updated with offline-ready setup instructions and model download commands.

Phase 4 – Final Optimisations (April 2026, final commits)

Date	Change
2026-04-12	Config: added CTranslate2 thread tuning; dual beam sizes; NLLB greedy flag.

2026-04-12	ASR: refined dual-path decode (beam=1 partial, beam=4 final); improved hallucination handling.
2026-04-12	Translation: use all CPU cores; greedy beam; drop stale partials to reduce latency.
2026-04-12	TTS: replaced <code>afplay</code> subprocess (macOS-only) with <code>sounddevice.OutputStream</code> (cross-platform); added TTS pre-warm at startup; fixed E2E latency bug.
2026-04-12	State: added <code>tts_play_ms</code> field to separate synthesis latency from playback duration.
2026-04-12	Segmenter: renamed internal queue; added partial-flush interval for live dashboard.
2026-04-12	Audio: minor cleanup (removed unused import).
2026-04-12	Main: cleaner model-loading sequence with startup status logging.
2026-04-12	Dashboard: exposed partial ASR/translation; per-stage latency breakdown visible in real time.
2026-04-12	feat: improved real-time pipeline accuracy and reliability across ASR/translation/TTS.
2026-04-12	chore: added platform-specific dependency profiles and setup docs for macOS and Linux CUDA.
2026-04-12	Improved Mac compatibility and pipeline fallbacks (Silero VAD, CTranslate2, TTS device).
2026-04-12	Improved latency reporting and translation responsiveness (stale-partial skip, cache hit).
2026-04-12	Added session reset flow to dashboard (“Reset Session” button) and pipeline (drain queues, increment session ID).

References

- [1] A. Radford, J. W. Kim, T. Xu, G. Brockman, C. McLeavey, and I. Sutskever, “Robust Speech Recognition via Large-Scale Weak Supervision,” in *Proc. 40th International Conference on Machine Learning (ICML)*, PMLR, 2023.
- [2] G. Klein and others, “Faster Whisper,” *GitHub Repository*, 2023. <https://github.com/SYSTRAN/faster-whisper>
- [3] G. Klein, Y. Kim, Y. Deng, J. Senellart, and A. Rush, “OpenNMT: Open-Source Toolkit for Neural Machine Translation,” in *Proc. ACL System Demonstrations*, 2017. (CTranslate2 is the optimised inference engine derived from this work.)
- [4] I. Silero Team, “Silero VAD: Pre-trained Enterprise-Grade Voice Activity Detectors,” *GitHub Repository*, 2021. <https://github.com/snakers4/silero-vad>
- [5] NLLB Team *et al.*, “No Language Left Behind: Scaling Human-Centered Machine Translation,” *arXiv:2207.04672*, 2022.
- [6] V. Pratap *et al.*, “Scaling Speech Technology to 1,000+ Languages,” *arXiv:2305.13516*, 2023.
- [7] J. Kim, J. Kong, and J. Son, “Conditional Variational Autoencoder with Adversarial Learning for End-to-End Text-to-Speech,” in *Proc. 38th ICML*, PMLR, 2021.

- [8] Seamless Communication *et al.*, “SeamlessM4T: Massively Multilingual and Multimodal Machine Translation,” *arXiv:2308.11596*, 2023.